این صورت ممکن است شرایط بن‌بست اتفاق بیفتد. در این محیط، سیستم می‌تواند ابزار زیر را فراهم کند:

- الگوریتمی که حالت سیستم را بررسی می‌کند تا تعیین کند که آیا بن‌بستی رخ داده است یا خیر؟
- الگوریتمی برای ترمیم از بن‌بست

در ادامه، جزئیات این نیازمندی‌ها را در سیستم‌هایی با نمونه منابع منفرد، در کنار سیستم‌هایی با تعدد نمونه‌های منابع بحث می‌کنیم. با این وجود، توجه داریم که طرح تشخیص و ترمیم بن‌بست سرباری لازم دارد که نه تنها هزینه‌های نگهداری اطلاعات لازم در زمان اجرا و اجرای الگوریتم تشخیص را شامل می‌شود، بلکه پتانسیل زیادی ذاتاً در ترمیم و رفع بن‌بست هدر می‌رود.

۸-۷-۱ نمونه‌های منفرد انواع منابع

اگر تمامی منابع فقط یک نمونه داشته باشند، در این صورت می‌توان الگوریتمی برای تشخیص بن‌بست تعریف کرد که گونه‌ای از گراف تخصیص منبع به نام گراف *انتظارکشی*[1] را به کار می‌برد. این گراف را از روی گراف تخصیص منبع با حذف گره‌های نوع منابع و ادغام یال‌های مناسب به‌دست می‌آوریم.

به بیان دقیق‌تر، وجود یک یال T_i به T_j در *گراف انتظارکشی* به این معناست که نخ T_i منتظر منبعی است که نخ T_j آن را در اختیار دارد. یک یال $T_i \rightarrow T_j$ در *گراف انتظارکشی* به وجود می‌آید اگر و تنها اگر، گراف تخصیص منبع متناظر آن، برای برخی منابع R_q، دو یال $T_i \rightarrow R_q$ و $R_q \rightarrow T_j$ را داشته باشد. ما در شکل ۸-۱۱، یک گراف تخصیص منبع و گراف *انتظارکشی* متناظر آن را ارائه می‌کنیم.

درست همانند قبل، سیستم در حالت بن‌بست واقع است اگر و تنها اگر، گراف *انتظارکشی* دارای دور باشد. سیستم برای تشخیص بن‌بست لازم دارد که یک گراف *انتظارکشی* نگهداری کرده و با تناوبی *الگوریتمی را احضار نماید* تا گراف را برای تشخیص دور پیمایش کند. الگوریتم تشخیص دور یک گراف به پیچیدگی عملیاتی از مرتبه‌ی n^2 نیاز دارد که در آن، n تعداد رئوس در گراف می‌باشد.

مجموعه‌ی ابزار BCC که در بخش ۲-۱۰-۴ بیان شد، ابزاری فراهم می‌کند که می‌تواند بن‌بست‌های بالقوه‌ی قفل‌های انحصار متقابل نخ‌های پوزیکس را در یک پردازه‌ی کاربر در حال اجرا بر روی یک سیستم لینوکس را تشخیص دهد. ابزار BCC deadlock_detector با درج پروب‌هایی که فراخوانی‌ها به توابع pthread_mutex_lock() و pthread_mutex_unlock() را ردیابی می‌کنند عمل می‌کند. موقعی که پردازه‌ی مشخص شده یک فراخوانی به هر کدام از توابع انجام می‌دهد، deadlock_detector یک گراف انتظارکشی از قفل‌های انحصار متقابل آن پردازه ساخته و احتمال بن‌بست را در صورتی که یک دوری در گراف تشخیص بدهد، گزارش می‌دهد.

1. Wait-for graph

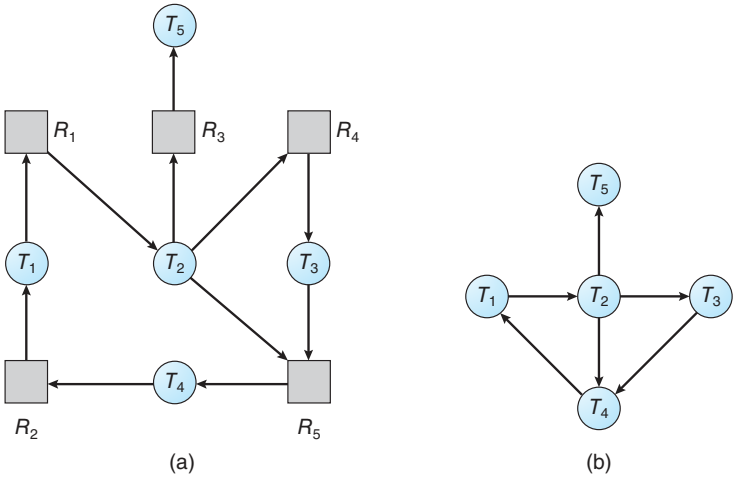
۸-۷-۲ تعدد نمونه‌های انواع منابع

طرح گراف *انتظارکشی* نمی‌تواند در یک سیستم تخصیص منبع با تعدد نمونه‌های هر یک از انواع منابع استفاده شود. اکنون به بیان الگوریتمی می‌پردازیم که می‌تواند برای تشخیص بن‌بست این قبیل سیستم‌ها به کار گرفته شود. این الگوریتم شبیه الگوریتم بانکدار (بخش ۸-۶-۳) از ساختمان‌داده‌های متعددی که در طول زمان تغییر می‌کنند استفاده می‌نماید که عبارتند از:

- **Available**: برداری به طول m، که تعداد منابع در دسترس هر یک از انواع منابع را نشان می‌دهد.
- **Allocation**: ماتریس n × m که تعداد هر یک از انواع منابع در اختیار فعلی هر یک از نخ‌ها را تعریف می‌کند.
- **Request**: ماتریس n × m که درخواست فعلی منابع هر یک از نخ‌ها را نشان می‌دهد. اگر Request[i][j] برابر k باشد، در این صورت نخ T_i، k نمونه‌ی دیگر از منبع نوع R_j را درخواست داده است.

رابطه‌ی ≤ میان دو بردار، همانند بخش ۸-۶-۳ تعریف می‌شود. باز هم برای سادگی، سطرها و ستون‌های ماتریس‌های Allocation و Request را بردار فرض می‌کنیم؛ برای رجوع به آنها نیز از $Allocation_i$ و $Request_i$ استفاده خواهیم کرد. الگوریتم تشخیصی که در اینجا مطرح می‌شود به سادگی، تمامی دنباله‌های تخصیص منابع ممکن را برای نخ‌هایی که تکمیل نشده‌اند بررسی می‌کند. این الگوریتم را با الگوریتم بانکدار بخش ۸-۶-۳ مقایسه کنید.

۱- بردارهای Work و Finish را به ترتیب، به طول m و n در نظر بگیرید. Work ← Available. اگر برای i = ۰ تا n-۱، $Allocation_i \neq 0$ باشد، آنگاه Finish[i] ← false، در غیر این صورت Finish[i] ← true.



شکل ۸-۱۱ (a) گراف تخصیص منبع. (b) گراف انتظارکشی متناظر.

۲-یک اندیس i چنان بیابید که دو شرط زیر را با هم داشته باشد:

a. $Finish[i] == false$
b. $Request_i \leq Work$.

اگر یک i با این شرایط نبود، برو به مرحله‌ی ۴.

۳- $Work \leftarrow Work + Allocation_i$، $Finish[i] \leftarrow true$ و برو به مرحله‌ی ۲.

۴-اگر برای برخی نها، $0 \leq i < n$، $Finish[i] == false$ بود، در این صورت سیستم در یک حالت بن‌بست واقع است. به علاوه، اگر $Finish[i] == false$ باشد، آنگاه نخ T_i در بن‌بست قرار دارد.

این الگوریتم برای تشخیص این که آیا سیستم در حال بن‌بست واقع است یا خیر، به پیچیدگی عملیاتی از مرتبه‌ی $m \times n^2$ نیاز دارد.

شاید تعجب کنید که چرا به محض این که در می‌یابیم، $Request_i \leq Work$ می‌باشد (در مرحله‌ی ۲، قسمت b)، منابع نخ T_i را پس می‌گیریم (در مرحله‌ی ۳). اکنون می‌دانیم که T_i فعلاً در بن‌بست نیست (چون $Request_i \leq Work$). بنابراین، یک منش خوش‌بینانه پیشه کرده و فرض می‌کنیم که T_i برای تکمیل کار محوله‌اش به منابع بیشتری نیاز نخواهد داشت؛ لذا، به زودی تمامی منابع در اختیار فعلی خود را به سیستم پس خواهد داد. اگر فرض ما نادرست باشد، امکان دارد بعداً بن‌بستی اتفاق بیفتد. این بن‌بست نیز در دور بعدی که الگوریتم تشخیص بن‌بست احضار می‌شود، قابل شناسایی خواهد بود.

برای روشن‌شدن این الگوریتم، سیستمی را با پنج نخ T_0 تا T_4 و سه نوع منبع A، B و C در نظر می‌گیریم. نمونه‌های منابع A، B و C، به ترتیب هفت، دو و شش نمونه می‌باشند. تصویر لحظه‌ای زیر، وضعیت فعلی سیستم را نمایش می‌دهد:

	Allocation			*Request*			*Available*		
	A	B	C	A	B	C	A	B	C
T_0	۰	۱	۰	۰	۰	۰	۰	۰	۰
T_1	۲	۰	۰	۲	۰	۲			
T_2	۳	۰	۳	۰	۰	۰			
T_3	۲	۱	۱	۱	۰	۰			
T_4	۰	۰	۲	۰	۰	۲			

ادعا می‌کنیم که سیستم در حالت بن‌بست نیست. در حقیقت، اگر الگوریتم خودمان را اجرا کنیم، دنباله‌ی $<T_0, T_2, T_3, T_1, T_4>$ را پیدا می‌کنیم که برای تمامی نها، $Finish[i] == true$ را نتیجه می‌دهد.

حال فرض کنید نخ T_2 یک نمونه‌ی دیگر از منبع نوع C را درخواست کند. ماتریس Request، به صورت زیر تغییر می‌کند:

Request

	A	B	C
T_0	۰	۰	۰
T_1	۲	۰	۲
T_2	۰	۰	۱
T_3	۱	۰	۰
T_4	۰	۰	۲

اکنون ادعا می‌کنیم که سیستم در حالت بن‌بست قرار دارد. اگرچه می‌توانیم منابع در اختیار نخ T_0 را پس بگیریم، ولی تعداد منابع موجود برای انجام درخواست‌های سایر نخ‌ها کافی نیست. بنابراین، یک بن‌بست شامل نخ‌های T_1، T_2، T_3 و T_4 به وجود می‌آید.

تشخیص بن‌بست با استفاده از رونوشت‌های نخ جاوا

اگرچه جاوا پشتیبانی صریحی برای تشخیص بن‌بست فراهم نمی‌کند، یک **رونوشت نخ**[1] می‌تواند برای تحلیل یک برنامه‌ی در حال اجر به‌منظور تعیین این که آیا یک بن‌بست وجود دارد یا خیر، استفاده شود. رونوشت نخ یک ابزار عیب‌یابی مفیدی است که یک تصویر لحظه‌ای از حالت‌های تمامی نخ‌های یک برنامه‌ی جاوا نمایش می‌دهد. رونوشت نخ‌های جاوا اطلاعات قفل‌گذاری را نیز نشان می‌دهد، اعم از این که یک نخ مسدود به‌انتظار کسب کدام یک از قفل‌هاست. موقعی که یک رونوشت نخ تولید می‌شود، JVM گراف انتظارکشی را برای تشخیص دور جستجو کرده، هرگونه بن‌بستی را که تشخیص داد، گزارش می‌نماید. برای تولید یک رونوشت نخ از یک برنامه‌ی در حال اجرا، فرمان زیر را از طریق خط فرمان وارد کنید:

```
Ctrl-L (UNIX, Linux, ormacOS)
Ctrl-Break (Windows)
```

ما در کد اصلی دانلود شده‌ی همراه این کتاب یک نمونه برنامه‌ی جاوا که در شکل ۸-۱ نشان داده شده تدارک دیده‌ایم و چگونگی تولید یک رونوشت نخ را که نخ‌های در بن‌بست جاوا را گزارش می‌دهد بیان می‌کنیم.

۸-۷-۳ استفاده از الگوریتم تشخیص

چه موقع می‌بایست الگوریتم تشخیص را احضار کنیم؟ جواب این سؤال، به دو عامل زیر بستگی دارد:

۱-معمولاً چند به وقت، یک بن‌بست اتفاق می‌افتد؟

۲-زمانی که بن‌بست اتفاق می‌افتد چه تعداد از نخ‌ها متأثر خواهند شد؟

اگر بن‌بست‌ها به کرات اتفاق می‌افتند، در این صورت بایستی الگوریتم تشخیص با تناوب بیشتری احضار شود. منابع در اختیار نخ‌های واقع در بن‌بست تا شکستن بن‌بست بیکار خواهند ماند. مضاف بر این، تعداد نخ‌های دور بن‌بست نیز می‌تواند افزایش یابد.

بن‌بست‌ها فقط زمانی اتفاق می‌افتند که یک نخ درخواستی می‌دهد که بلافاصله قابل اعطا نیست. این درخواست شاید آخرین درخواستی است که دور زنجیره‌ی نخ‌های منتظر را کامل

1. Thread dump

می‌کند. در حد نهایت، الگوریتم تشخیص بن‌بست را زمانی احضار می‌کنیم که نتوان یک درخواست را بلافاصله اعطا کرد. در این حالت، نه تنها می‌توانیم نخ‌های واقع در بن‌بست را شناسایی کنیم، بلکه نخی که «باعث» بن‌بست بوده نیز قابل شناسایی است. (در حقیقت، هر یک از نخ‌هایی که در بن‌بست قرار دارند، اتصالی از دور ایجاد شده در گراف منبع می‌باشند که پیوستن تمامی آنها موجب به وجود آمدن بن‌بست می‌شود.) اگر انواع منابع متفاوت زیاد باشند، یک درخواست می‌تواند باعث ایجاد دورهای زیادی در گراف منبع شود که هر یک از این دورها با اخیرترین درخواست کامل شده‌اند و نخ «باعث» آن قابل شناسایی است.

البته، احضار الگوریتم تشخیص بن‌بست با هر درخواست می‌تواند از نظر زمان محاسبات، سربار قابل توجهی به بار آورد. روش ساده و کم هزینه آن است که، الگوریتم را با تناوب کمتری احضار کنیم—برای مثال، یک بار در هر ساعت یا هر بار که بهره‌وری پردازنده از ۴۰ درصد پایین‌تر بیاید. (بالاخره، بن‌بست بازده سیستم را ناکار کرده و افت بهره‌وری پردازنده را در پی دارد.) اگر الگوریتم تشخیص در نقاط زمانی دلخواهی احضار شود، گراف منبع می‌تواند دورهای زیادی داشته باشد. در این حالت، به طور کلی نمی‌توانیم بگوییم که کدام یک از انبوه نخ‌های واقع در بن‌بست باعث پیدایش بن‌بست بوده‌اند.

مدیریت بن‌بست در پایگاه‌داده‌ها

سیستم‌های پایگاه‌داده یک نمایش مفیدی از چگونگی مدیریت بن‌بست هر دو نرم‌افزار متن‌باز و تجاری فراهم می‌کنند. به‌روزرسانی‌های یک پایگاه‌داده می‌تواند به‌صورت **تراکنش‌هایی** انجام شود و نوعاً برای تضمین جامعیت داده‌ها از قفل استفاده می‌شود. یک تراکنش ممکن است مشمول قفل‌های متعددی باشد، لذا تعجبی ندارد که در یک پایگاه‌داده‌ای با تراکنش‌های متعدد همروند احتمال بن‌بست وجود داشته باشد. اکثر سیستم‌های پایگاه‌داده‌ی تراکنشی به‌منظور مدیریت بن‌بست، یک راهکار تشخیص و ترمیم از بن‌بست کار می‌گذارند. کارگزار پایگاه‌داده به‌صورت متناوب در گراف انتظارکشی دنبال دور می‌گردد تا بن‌بست میان یک مجموعه تراکنش را تشخیص بدهد. موقعی که بن‌بست تشخیص داده شد، یک قربانی انتخاب شده و تراکنش برکنار می‌گردد و به عقب کشیده می‌شود، قفل‌های نگه داشته شده توسط تراکنش قربانی رها شده و تراکنش‌های باقیمانده‌ی بن‌بست از بن‌بست رها می‌شوند. همین که تراکنش‌های باقیمانده از سر گرفته شدند، تراکنش برکنار شده دومرتبه صادر می‌شود. انتخاب یک تراکنش قربانی به سیستم پایگاه‌داده بستگی دارد؛ برای نمونه، MySQL تلاش می‌کند تا تراکنش‌هایی را انتخاب کند که تعداد ردیف‌های درج شده، به‌روزشده، یا حذفی را کمینه می‌سازد.

۸-۸ ترمیم از بن‌بست

موقعی که الگوریتم تشخیص، وجود بن‌بست را معین کرد، روش‌های متعددی برای برخورد با آن وجود دارد. یک روش این است که وقوع بن‌بست را به اپراتور خبر داده و اجازه دهیم تا اپراتور به صورت دستی با مسأله‌ی بن‌بست برخورد کند. روش دیگر آن است که اجازه دهیم تا سیستم به صورت خودکار از حالت بن‌بست **ترمیم** یابد. برای شکستن بن‌بست دو انتخاب وجود دارد. راه‌حل ساده آن است که یک یا چند نخ را کنار بگذاریم تا انتظار چرخشی شکسته شود. گزینه‌ی دیگر آن است که برخی منابع را از یک یا چند نخ واقع در بن‌بست پس بگیریم.

۸-۸-۱ ختم پردازه و نخ

زمانی که بن‌بست را با برکناری یک پردازه یا نخ رفع می‌کنیم، یکی از دو روش زیر را به کار می‌بریم. در هر دو روش، سیستم تمامی منابع در اختیار پردازه‌های برکنار شده را پس می‌گیرد.

- **برکناری تمامی پردازه‌های بن‌بست**: روشن است که این روش دور بن‌بست را خواهد شکست، اما با هزینه‌ای بالا. ممکن است پردازه‌هایی که در بن‌بست افتاده‌اند محاسبات طولانی‌مدتی داشته باشند که با این کار، محاسبات جزئی آنها هدر رفته و احتمالاً در ادامه، به ناچار از نو محاسبه خواهند شد.
- **هر بار یک پردازه برکنار می‌شود تا این که دور بن‌بست حذف شود**: این روش، سربار قابل توجهی به دنبال دارد، چون با برکناری هر یک از پردازه‌های واقع در دور بن‌بست بایستی الگوریتم تشخیص بن‌بست احضار شود تا تعیین کند که آیا هنوز هم پردازه‌ها در بن‌بست قرار دارند یا خیر.

برکناری یک پردازه ممکن است به این سادگی هم نباشد. اگر پردازه در وسط عملیات به روزکردن یک فایل باشد، ختم آن باعث می‌شود که فایل در یک حالت نادرست رها شود. به طور مشابه، اگر پردازه در وسط به‌روزرسانی داده‌های مشترکی باشد در حالی که یک قفل انحصار متقابل را نگه داشته است، سیستم می‌بایست وضعیت قفل را بازیابی کند تا در دسترس باشد، هرچند که نمی‌توان راجع به جامعیت داده‌های مشترک هیچ تضمینی داد.

اگر از روش ختم جزئی استفاده شود، در این صورت با داشتن یک مجموعه پردازه که در بن‌بست قرار دارند، بایستی تعیین شود که بهتر است کدام پردازه (یا پردازه‌ها) را برای شکستن بن‌بست از میان برداریم. این انتخاب، مشابه تصمیم‌های زمانبندی پردازنده، یک نوع اتخاذ سیاست محسوب می‌شود. اساس این سؤال اقتصادی است؛ بایستی پردازه‌هایی را برکنار کنیم که ختم آنها حداقل هزینه را در پی داشته باشد. متأسفانه، اصطلاح *حداقل* هزینه هم چندان روشن نیست. عوامل زیادی هستند که می‌توانند روی انتخاب یک پردازه جهت برکناری تأثیرگذار باشند که عبارتند از:

۱-پردازه چه الویتی دارد
۲-پردازه چه مدت محاسبات داشته و چه میزان محاسبات دیگر لازم دارد تا کار محوله‌اش را تکمیل نماید
۳-پردازه به چه تعداد و از چه نوع منابعی استفاده کرده است (برای مثال، آیا می‌توان منابع را به سادگی پس گرفت یا خیر)
۴-چه تعداد منابع دیگر به منظور تکمیل کار پردازه لازم است
۵-چه تعداد پردازه لازم است که خاتمه داده شوند

۸-۸-۲ پس‌گیری منبع

در صورتی که برای شکستن بن‌بست‌ها از پس‌گیری منبع استفاده کنیم، برخی منابع پردازه‌های واقع در بن‌بست را پی در پی پس‌گرفته و در اختیار سایر پردازه‌های واقع در بن‌بست قرار

می‌دهیم تا این که دور بن‌بست شکسته شود.

اگر برای برخورد با بن‌بست‌ها از روش پس‌گیری منابع استفاده شود، در این صورت لازم است که به ملاحظات سه‌گانه‌ی زیر دقت کنیم:

۱- **انتخاب قربانی**: کدام یک از منابع و کدام یک از پردازه‌ها پس‌گرفته می‌شوند؟ درست همانند ختم پردازه، بایستی پس‌گیری را به ترتیبی انجام بدهیم که هزینه کمینه شود. عوامل هزینه در اینجا می‌تواند تعداد منابعی باشد که در اختیار پردازه‌های واقع در بن‌بست می‌باشند و نیز مدت زمانی که پردازه‌های واقع در بن‌بست تاکنون صرف کرده‌اند.

۲- **عقب‌کشی**: اگر منبعی را از یک پردازه پس بگیریم، بایستی با آن پردازه چه کار کنیم؟ روشن است که پردازه دیگر نمی‌تواند اجرای معمول خود را ادامه بدهد؛ چون برخی منابع مورد نیاز خود را از دست داده است. این پردازه را بایستی به حالت امنی به عقب بکشیم و دوباره از آن حالت آغاز کنیم.

به طور کلی، چون تعیین این که چه حالتی امن خواهد بود مشکل است، ساده‌ترین راه‌حل آن است که عقب‌کشی را کامل انجام دهیم: پردازه را برکنار کرده و دوباره از نو شروع کنیم. هرچند سودمندتر است که پردازه را فقط تا جایی عقب بکشیم که بن‌بست شکسته می‌شود، این روش هم مستلزم آن است که سیستم اطلاعات بیشتری را در رابطه با حالت تمامی پردازه‌های در حال اجرا نگهداری کند.

۳- **قطحی‌زدگی**: از کجا مطمئن شویم که قطحی‌زدگی اتفاق نخواهد افتاد؟ یعنی، چگونه می‌توان تضمین کرد که همواره منابع پردازه‌های خاصی پس‌گرفته نمی‌شوند؟

در سیستمی که انتخاب قربانی عمدتاً بر اساس عوامل هزینه صورت می‌گیرد، امکان دارد شرایطی به وجود بیاید که همواره پردازه‌ای خاص به عنوان قربانی انتخاب شود. پیامد این کار آن است که چنین پردازه‌ای هرگز موفق نمی‌شود کار محوله‌اش را تکمیل کند، بایستی هر سیستم عملی، برنامه‌ای برای شرایط قحطی‌زدگی داشته باشد. روشن است، بایستی اطمینان بدهیم که تعداد دفعاتی که یک پردازه می‌تواند به عنوان قربانی انتخاب شود محدود (کم) است. راه‌حل مرسوم آن است که تعداد دفعات عقب‌کشی را نیز یک عامل هزینه به حساب بیاوریم.

۸-۹ خلاصه

- بن‌بست در یک مجموعه‌ای از پردازه‌ها پیش می‌آید که هر پردازه در مجموعه به‌انتظار رویدادی است که تنها می‌تواند توسط پردازه‌ی دیگری از همان مجموعه اتفاق بیفتد.
- چهار شرط برای بن‌بست لازمند که عبارتند از: (۱) انحصار متقابل، (۲) گرفتن و منتظر ماندن، (۳) نبود پس‌گیری، و (انتظار چرخشی). بن‌بست فقط موقعی امکان‌پذیر است که تمامی چهار شرط برقرار باشند.
- بن‌بست‌ها می‌توانند با گراف‌های تخصیص منبع مدل‌سازی شوند که در آن‌ها، یک